GENERATIVE AI & LLMS

LangChain

Compose models, prompts, retrievers, memory, tools and agents into production LLM apps with LCEL, RAG and LangGraph.

langchain · lcel · rag · agents · langgraph · llm · python

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What LangChain Is & Why It Matters
2. The LangChain Ecosystem & Packages
3. Setup & Getting Started
4. Core Abstraction 1 — Chat Models & Messages
5. Core Abstraction 2 — Prompt Templates
6. Core Abstraction 3 — Output Parsers & Structured Output
7. LCEL — The Runnable Composition Model
8. Chains & Resilience Patterns
9. Retrieval & RAG with LangChain
10. Memory & Conversation History
11. Tools & Agents
12. LangGraph — Stateful Orchestration
13. Integrations Landscape
14. Cheat Sheet & Comparison
15. Best Practices & Common Pitfalls
16. Interview Questions & Answers
17. Further Learning

What LangChain Is & Why It Matters

LangChain is a framework for building applications powered by large language models (LLMs). It gives you composable building blocks — models, prompts, parsers, retrievers, memory, tools and agents — and a declarative way to wire them together into reliable pipelines.

Calling an LLM API once is trivial. Building a *product* around it is not: you need prompt templating, structured output, retrieval over your own data, conversation memory, tool-calling, retries, streaming, tracing, and the ability to swap one provider for another without rewriting everything. LangChain's job is to make those concerns reusable and provider-agnostic.

> **Mental model**
> Think of LangChain as "LEGO for LLM apps". Each brick has a uniform interface (`Runnable`), so bricks snap together with the pipe operator and the whole assembly streams, batches, and runs async for free.

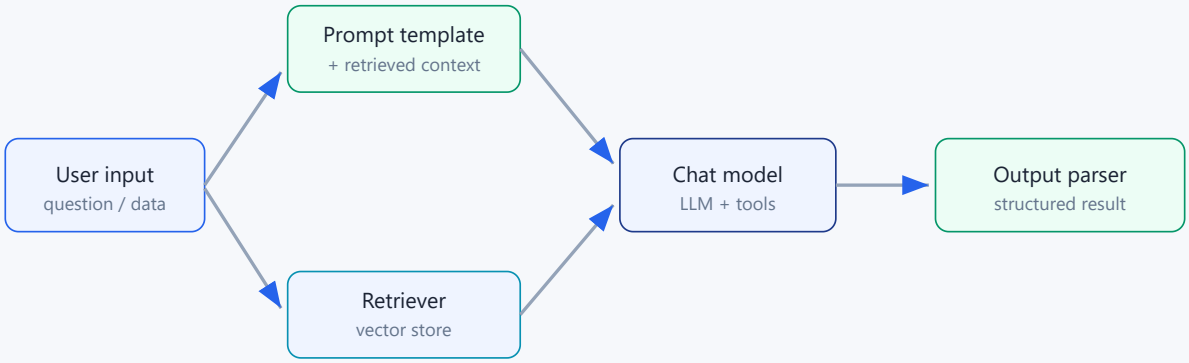


Fig 1. A typical LangChain RAG pipeline: input fans out to a retriever and prompt, flows into the model, then through a parser.

PROVIDER-AGNOSTIC

Swap `langchain-openai` for `langchain-anthropic` or a local model by changing one line. The `Runnable` interface is identical.

COMPOSABLE

LCEL lets you build pipelines with `|`. Every pipeline is itself a Runnable, so you compose pipelines of pipelines.

PRODUCTION-READY

Streaming, async, batching, retries, fallbacks and tracing (LangSmith) come built in rather than bolted on.

The LangChain Ecosystem & Packages

Since v0.1 LangChain split into focused packages so you install only what you use. As of the 1.x era the layout is stable:

Package	Contains	Install
`langchain-core`	Base abstractions: `Runnable`, messages, prompts, output parsers, tools. Zero heavy deps.	`pip install langchain-core`
`langchain`	Higher-level chains, retrieval helpers, agent constructors.	`pip install langchain`
`langchain-openai` / `-anthropic` / …	Provider integrations (one package per provider).	`pip install langchain-openai`
`langgraph`	Stateful, graph-based orchestration for agents and multi-step workflows.	`pip install langgraph`
`langchain-community`	Community-maintained integrations (vector stores, loaders, tools).	`pip install langchain-community`
`langsmith`	Tracing, evaluation and observability SDK.	`pip install langsmith`

✦ **Rule of thumb**

Import stable abstractions from `langchain_core`, models from provider packages, and reach for `langgraph` the moment your logic has loops, branches, or persistent state.

Setup & Getting Started

BASH

```
# Create an isolated environment
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate

# Core + a provider + langgraph for agents
pip install langchain langchain-core langchain-openai langchain-anthropic langgraph

# Set your API keys (never hard-code them)
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."

# Optional: turn on tracing to see every step in LangSmith
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="lsv2_..."
```

PYTHON

```python
# hello.py — the smallest possible LangChain program
from langchain.chat_models import init_chat_model

# init_chat_model picks the right provider package from the model string
model = init_chat_model("gpt-4o-mini", model_provider="openai", temperature=0)

response = model.invoke("Explain vector embeddings in one sentence.")
print(response.content)
```

> **init_chat_model**
> This helper is the modern, provider-neutral entry point. Pass `"anthropic:claude-sonnet-4-5"` or `"ollama:llama3.1"` and nothing else in your code changes.

Core Abstraction 1 — Chat Models & Messages

Modern LLMs are **chat models**: they consume a list of typed messages and return an `AIMessage`. LangChain normalizes every provider to the same message classes.

Message class	Role	Purpose
`SystemMessage`	system	Instructions / persona that steer the whole conversation.
`HumanMessage`	user	Input from the end user.
`AIMessage`	assistant	Model output; may carry `tool_calls`.
`ToolMessage`	tool	Result of running a tool, fed back to the model.

PYTHON

```python
from langchain_core.messages import SystemMessage, HumanMessage

messages = [
    SystemMessage("You are a terse senior engineer. Answer in ≤ 3 bullet points."),
    HumanMessage("When should I use a message queue?"),
]

ai = model.invoke(messages)
print(ai.content)
print(ai.usage_metadata)   # {'input_tokens': 29, 'output_tokens': 54, ...}
```

Common model configuration

Parameter	Effect
`temperature`	0 = deterministic/factual, higher = more creative/varied.
`max_tokens`	Caps the length of the response.
`timeout` / `max_retries`	Resilience against slow or flaky API calls.
`model_kwargs`	Pass provider-specific knobs not surfaced directly.

Core Abstraction 2 — Prompt Templates

Hard-coding prompts does not scale. `PromptTemplate` and `ChatPromptTemplate` turn prompts into reusable, parameterized objects with typed input variables.

PYTHON

```python
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant that explains {topic} to a {level} audienc
    ("human", "{question}"),
])

# .invoke fills the variables and returns a list of messages
msgs = prompt.invoke({"topic": "databases", "level": "beginner",
                      "question": "What is an index?"})
print(msgs.to_messages())
```

Use `MessagesPlaceholder` to splice in a running chat history or few-shot examples:

PYTHON

```python
from langchain_core.prompts import MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a coding tutor."),
    MessagesPlaceholder("history"),   # list of previous messages injected here
    ("human", "{input}"),
])
```

Core Abstraction 3 — Output Parsers & Structured Output

Raw text is hard to consume programmatically. Parsers convert model output into typed Python objects. The modern, most reliable path is `.with_structured_output()`, which uses the provider's native tool/JSON-schema support.

PYTHON

```python
from pydantic import BaseModel, Field

class MovieReview(BaseModel):
    title: str = Field(description="The movie's title")
    rating: int = Field(ge=1, le=10, description="Score from 1 to 10")
    pros: list[str]
    cons: list[str]

structured = model.with_structured_output(MovieReview)
review = structured.invoke("Review the film Inception.")
print(review.rating, review.pros)   # → 9 ['inventive premise', ...]
# `review` is a validated MovieReview instance, not a string
```

STROUTPUTPARSER

Extracts `.content` as a plain string — the most common final step in a text chain.

JSONOUTPUTPARSER

Parses JSON and can stream partial objects as they arrive.

✦ **Prefer schemas over regex**

`with_structured_output` with a Pydantic model gives you validation, type hints, and editor autocomplete — and it fails loudly when the model drifts.

LCEL — The Runnable Composition Model

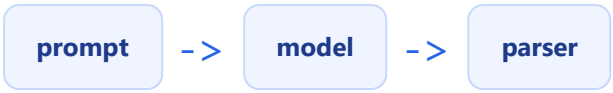
LCEL (LangChain Expression Language) is the heart of the framework. Every component implements the `Runnable` interface, so you compose them with the pipe operator `|` exactly like Unix pipes.

PYTHON

```python
from langchain_core.output_parsers import StrOutputParser

# prompt → model → parser, as one streaming, async-ready Runnable
chain = prompt | model | StrOutputParser()

chain.invoke({"topic": "networking", "level": "expert",
              "question": "Explain TCP backpressure."})
```



Because the composed object is *itself* a Runnable, you get a uniform execution surface for free:

Method	What it does
`invoke(x)`	Run once on a single input.
`batch([x1, x2])`	Run many inputs concurrently.
`stream(x)`	Yield output tokens/chunks as they are produced.
`ainvoke` / `abatch` / `astream`	Async equivalents of all of the above.

PYTHON

```python
# Streaming tokens to a UI
for chunk in chain.stream({"topic": "AI", "level": "beginner",
                           "question": "What is a transformer?"}):
    print(chunk, end="", flush=True)

# Concurrent batch
chain.batch([{"topic": "git", "level": "beginner", "question": q}
             for q in ["What is a commit?", "What is a rebase?"]])
```

RunnablePassthrough, RunnableParallel & RunnableLambda

These primitives build non-linear graphs. `RunnableParallel` runs branches concurrently; `RunnablePassthrough` forwards the original input; `RunnableLambda` wraps any plain function as a Runnable.

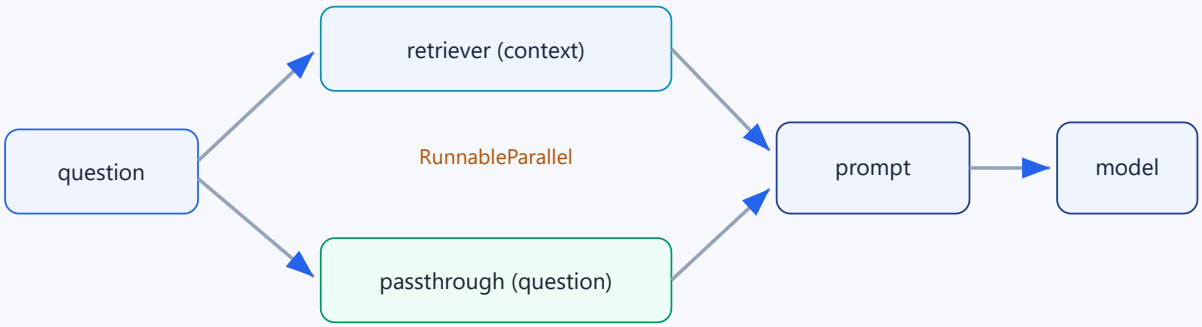


Fig 2. A `RunnableParallel` dict fans the input into named branches that run concurrently, then merges into a single dict for the next step.

PYTHON

```python
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# dict literals become RunnableParallel automatically inside a chain
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
rag_chain.invoke("What is our refund policy?")
```

Chains & Resilience Patterns

A "chain" is just a composed Runnable. Beyond linear flows, LCEL offers production features declaratively:

PYTHON

```
# Fallbacks: if the primary model errors, transparently try a backup
robust = model.with_fallbacks([backup_model])

# Retries with exponential backoff
resilient = chain.with_retry(stop_after_attempt=3)

# Attach config (tags, metadata) for tracing and run-time overrides
tagged = chain.with_config({"tags": ["prod", "support-bot"]})

# Bind tools or stop sequences onto a model
model_with_tools = model.bind_tools([search, calculator])
```

✓ **Why this is powerful**

Retries, fallbacks, and streaming are cross-cutting concerns. In LCEL they are composable wrappers, so you add resilience without rewriting business logic.

Retrieval & RAG with LangChain

Retrieval-Augmented Generation grounds the model in your own documents, cutting hallucination and letting the model answer questions about private or recent data it was never trained on.

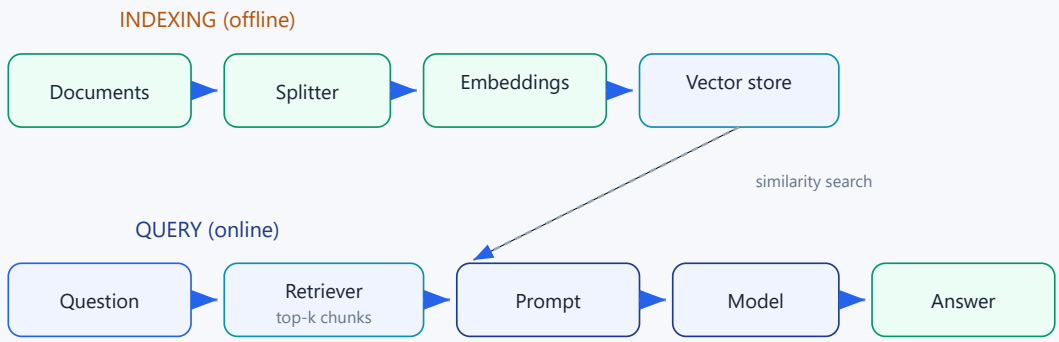


Fig 3. RAG has two phases: index documents into a vector store offline, then retrieve the most relevant chunks at query time to ground the model.

Building a RAG pipeline end to end

PYTHON

```python
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

# 1) Load
docs = WebBaseLoader("https://docs.example.com/handbook").load()

# 2) Split into overlapping chunks (keeps context across boundaries)
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)

# 3) Embed + store
store = InMemoryVectorStore.from_documents(chunks, OpenAIEmbeddings())

# 4) Turn the store into a retriever (returns Documents for a query)
retriever = store.as_retriever(search_kwargs={"k": 4})

# 5) Compose the answer chain
from langchain_core.prompts import ChatPromptTemplate
rag_prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY this context. If unsure, say you don't know.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | rag_prompt | model | StrOutputParser()
)
print(chain.invoke("How many vacation days do new hires get?"))
```

RAG knob	Trade-off
`chunk_size`	Larger = more context per chunk but noisier and more tokens.
`chunk_overlap`	Prevents facts being split across a boundary; costs duplication.
`k` (top-k)	More chunks = higher recall but risk of distraction + cost.
Retriever type	Similarity, MMR (diversity), or hybrid (keyword + vector).

Memory & Conversation History

LLMs are stateless — each call sees only what you pass. "Memory" means persisting prior turns and replaying them. The legacy `ConversationBufferMemory` classes are deprecated; the modern approach uses `RunnableWithMessageHistory` or, preferably, LangGraph's built-in persistence.

PYTHON

```python
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

store = {}
def get_history(session_id: str):
    return store.setdefault(session_id, InMemoryChatMessageHistory())

chat = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
]) | model

with_memory = RunnableWithMessageHistory(
    chat, get_history,
    input_messages_key="input", history_messages_key="history",
)

cfg = {"configurable": {"session_id": "user-42"}}
with_memory.invoke({"input": "My name is Ada."}, config=cfg)
with_memory.invoke({"input": "What is my name?"}, config=cfg)  # → "Ada"
```

⚠ **Context window is finite**

Replaying unbounded history will blow your token budget and cost. Trim with `trim_messages`, summarize older turns, or window to the last N messages.

Tools & Agents

A tool is a function the model can choose to call. An agent is an LLM in a loop: it reasons, decides which tool to call, observes the result, and repeats until it can answer. This is how LLMs reach beyond their training data — searching, calculating, querying databases, hitting APIs.

PYTHON

```python
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Return the current weather for a city. Use for any weather question."""
    # In reality you'd call a weather API here
    return f"It is 22C and sunny in {city}."

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b
```

✦ **The docstring is the API**

The model decides whether to call a tool from its name, type hints and docstring. Write them as if documenting for a junior engineer — clear descriptions dramatically improve tool selection.

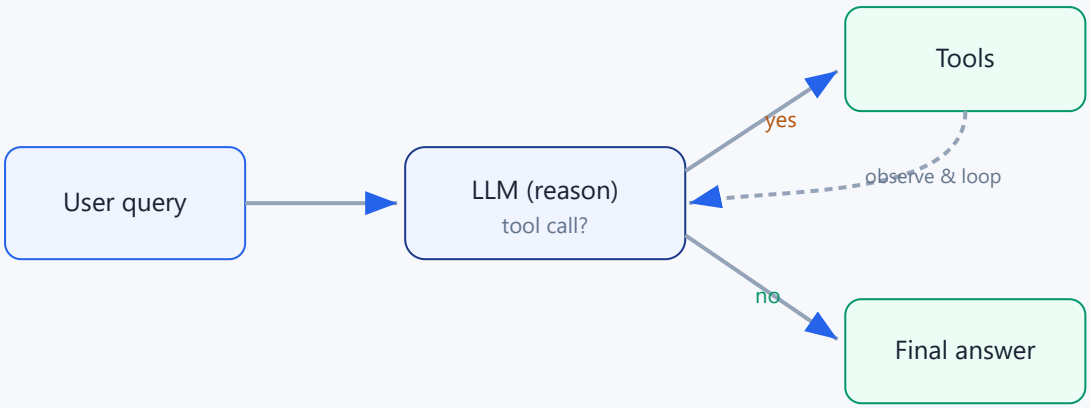


Fig 4. The agent loop: the LLM reasons, optionally calls a tool, observes the result, and loops until it produces a final answer.

Creating an agent (LangGraph)

PYTHON

```python
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(
    model="openai:gpt-4o",
    tools=[get_weather, multiply],
    prompt="You are a precise assistant. Use tools when needed.",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Weather in Oslo, and what is 23*19?"}]}
)
print(result["messages"][-1].content)
```

Where agents live now

As of LangChain 1.x the recommended agent runtime is `langgraph`. The older `AgentExecutor` / `initialize_agent` helpers still exist but are considered legacy — new code should use `create_react_agent` or a custom graph.

LangGraph — Stateful Orchestration

When your logic has branches, loops, human-in-the-loop pauses, or persistent state, a linear LCEL chain is not enough. LangGraph models your app as a **state graph**: nodes mutate a shared state object, edges (including conditional ones) decide what runs next.

PYTHON

```python
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver

class State(TypedDict):
    # add_messages reducer appends rather than overwrites
    messages: Annotated[list, add_messages]

def call_model(state: State):
    return {"messages": [model.invoke(state["messages"])]}

builder = StateGraph(State)
builder.add_node("chat", call_model)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)

# A checkpointer persists state per thread → durable memory + resumption
graph = builder.compile(checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "conv-1"}}
graph.invoke({"messages": [("user", "Hi, I'm Grace.")]}, cfg)
graph.invoke({"messages": [("user", "Who am I?")]}, cfg)  # remembers Grace
```

NODES

Plain functions (or Runnables) that read state and return partial updates.

CONDITIONAL EDGES

A router function inspects state and picks the next node — this is how you build loops and branches.

CHECKPOINTERS

Persist state per `thread_id` for memory, time-travel, and human-in-the-loop interrupts.

Integrations Landscape

Category	Examples
Chat models	OpenAI, Anthropic, Google Gemini, AWS Bedrock, Azure, Mistral, Ollama (local)
Embeddings	OpenAI, Cohere, HuggingFace, Voyage, local sentence-transformers
Vector stores	Chroma, Pinecone, Weaviate, Qdrant, pgvector, FAISS, Milvus
Document loaders	PDF, web, Notion, S3, Google Drive, SQL, Confluence (100s via community)
Tools	Tavily/web search, Python REPL, SQL, requests, plus any MCP server
Observability	LangSmith (tracing, evals, datasets, prompt management)

✦ **MCP**

LangChain can consume Model Context Protocol servers via `langchain-mcp-adapters`, turning any MCP server's tools into LangChain tools an agent can call.

Cheat Sheet & Comparison

Task	Snippet
Init any model	`init_chat_model("anthropic:claude-sonnet-4-5")`
Build a chain	`prompt | model | StrOutputParser()`
Structured output	`model.with_structured_output(MySchema)`
Stream	`for c in chain.stream(x): ...`
Parallel branches	`RunnableParallel(a=r1, b=r2)`
Retriever	`store.as_retriever(search_kwargs={"k":4})`
Define a tool	`@tool def f(x: int) -> int: ...`
Agent	`create_react_agent(model, tools)`
Fallback	`model.with_fallbacks([backup])`

When to use what

Need	Reach for
Fixed, linear transform (prompt → model → parse)	LCEL chain (`|`)
Grounding in private docs	Retriever + RAG chain
Model must take actions / call APIs	Tools + agent
Branches, loops, persistence, human-in-the-loop	LangGraph
Multi-turn conversation	LangGraph checkpointer / `RunnableWithMessageHistory`

Best Practices & Common Pitfalls

✓ **Do**

Pin package versions, set `temperature=0` for extraction/classification, use `with_structured_output` for anything a program consumes, and enable LangSmith tracing early — debugging LLM apps blind is painful.

✦ **Do**

Keep prompts in templates (version them), stream to the UI for perceived speed, and cap history/context with trimming or summarization.

⚠ **Watch out**

Over-large `k` or chunk sizes dilute relevance and inflate cost. Measure retrieval quality (recall/precision) before blaming the model.

✕ **Avoid**

Never pass unsanitized tool output or user input straight into shell/SQL/eval tools — prompt injection is real. Scope tool permissions, validate arguments, and add human approval for destructive actions.

⚠ **Deprecations**

Avoid legacy `LLMChain`, `ConversationChain`, and `initialize_agent` in new code. The modern stack is LCEL + LangGraph.

Interview Questions & Answers

What problem does LangChain solve that calling an LLM API directly does not?

It standardizes the surrounding plumbing: provider-agnostic model/message interfaces, reusable prompt templates, structured output, retrieval, memory, tool-calling, and production concerns (streaming, batching, async, retries, fallbacks, tracing) — all behind the uniform `Runnable` interface so components compose.

What is LCEL and why is the Runnable interface central?

LCEL (LangChain Expression Language) lets you compose components with the `|` operator. Every component is a `Runnable` exposing `invoke/batch/stream` plus async variants. Because a composed chain is itself a Runnable, you get streaming, concurrency, and resilience uniformly and can nest chains arbitrarily.

Explain RAG and why chunking and overlap matter.

RAG retrieves relevant documents and injects them into the prompt so the model answers from grounded, current, private data instead of parametric memory. Documents are split into chunks sized to balance context vs. noise and token cost; overlap prevents a fact from being severed at a chunk boundary, preserving continuity.

How does structured output work, and why prefer it over parsing text?

`with_structured_output(schema)` leverages the provider's native tool/JSON-schema support to return a validated Pydantic object. It gives type safety, validation, and autocomplete, and fails loudly when output drifts — far more robust than regex or hand-written JSON parsing.

What is the difference between a chain and an agent?

A chain is a predetermined sequence of steps — the control flow is fixed by you. An agent lets the LLM decide the control flow at runtime: it chooses which tools to call and when to stop, looping over reason→act→observe. Chains are predictable and cheap; agents are flexible but less deterministic and harder to constrain.

When would you choose LangGraph over a plain LCEL chain?

When you need cycles, conditional branching, persistent state across turns, human-in-the-loop approval, or resumable/durable execution. LCEL excels at linear DAG-style pipelines; LangGraph models stateful, cyclic workflows with checkpointing.

How do you give an LLM memory if models are stateless?

Persist prior messages and replay them on each call via a `MessagesPlaceholder`. Modern options are `RunnableWithMessageHistory` keyed by session, or a LangGraph checkpointer keyed by `thread_id`. Because context is finite, trim, window, or summarize older turns.

What makes a tool well-designed for an agent?

A clear name, precise type hints, and a descriptive docstring — the model selects tools from these. Keep each tool single-purpose, validate arguments, return concise results, and scope permissions. Vague docstrings are the number-one cause of wrong or missed tool calls.

What is prompt injection and how do you mitigate it in a tool-using agent?

Malicious instructions embedded in user input or retrieved/tool content that hijack the model into unintended actions. Mitigations: treat retrieved content as untrusted, never feed raw output into

shell/SQL/eval, constrain and validate tool arguments, scope credentials, require human approval for destructive operations, and add guardrail/validation layers.

How do you make a LangChain app production-resilient?

Use `with_retry` for transient failures, `with_fallbacks` for provider outages, timeouts and token caps, caching for repeated calls, streaming for UX, and LangSmith tracing/evals to catch regressions. Pin versions and separate prompt templates for versioning.

What does `RunnableParallel` (dict in a chain) do?

It runs multiple branches concurrently on the same input and merges their outputs into a dict. A dict literal placed in an LCEL pipeline is coerced into a `RunnableParallel` — the common pattern being `{"context": retriever, "question": RunnablePassthrough()}` feeding a prompt.

How do embeddings and vector stores enable retrieval?

An embedding model maps text to a high-dimensional vector where semantic similarity ≈ geometric closeness. The vector store indexes these vectors; at query time the question is embedded and a nearest-neighbor search returns the most semantically similar chunks, regardless of exact keyword overlap.

Further Learning

- Build the canonical RAG app end to end, then swap the vector store (Chroma → pgvector) to feel the provider-agnostic design.
- Rebuild the same assistant as a LangGraph graph with a checkpointer; add a conditional edge and a human-in-the-loop interrupt.
- Instrument everything with **LangSmith**: traces, a small eval dataset, and an LLM-as-judge scorer.
- Study advanced retrieval: MMR, hybrid search, re-ranking, parent-document and self-query retrievers.
- Explore multi-agent patterns in LangGraph (supervisor, hierarchical teams) and connect an MCP server via `langchain-mcp-adapters`.
- Read the official LangChain and LangGraph docs and the conceptual guides on agents, retrieval, and streaming.